

رسالة محمد



یادگیری ماشین

مدرس: محمدرضا محمدی

بهار ۱۴۰۲

درخت تصمیم

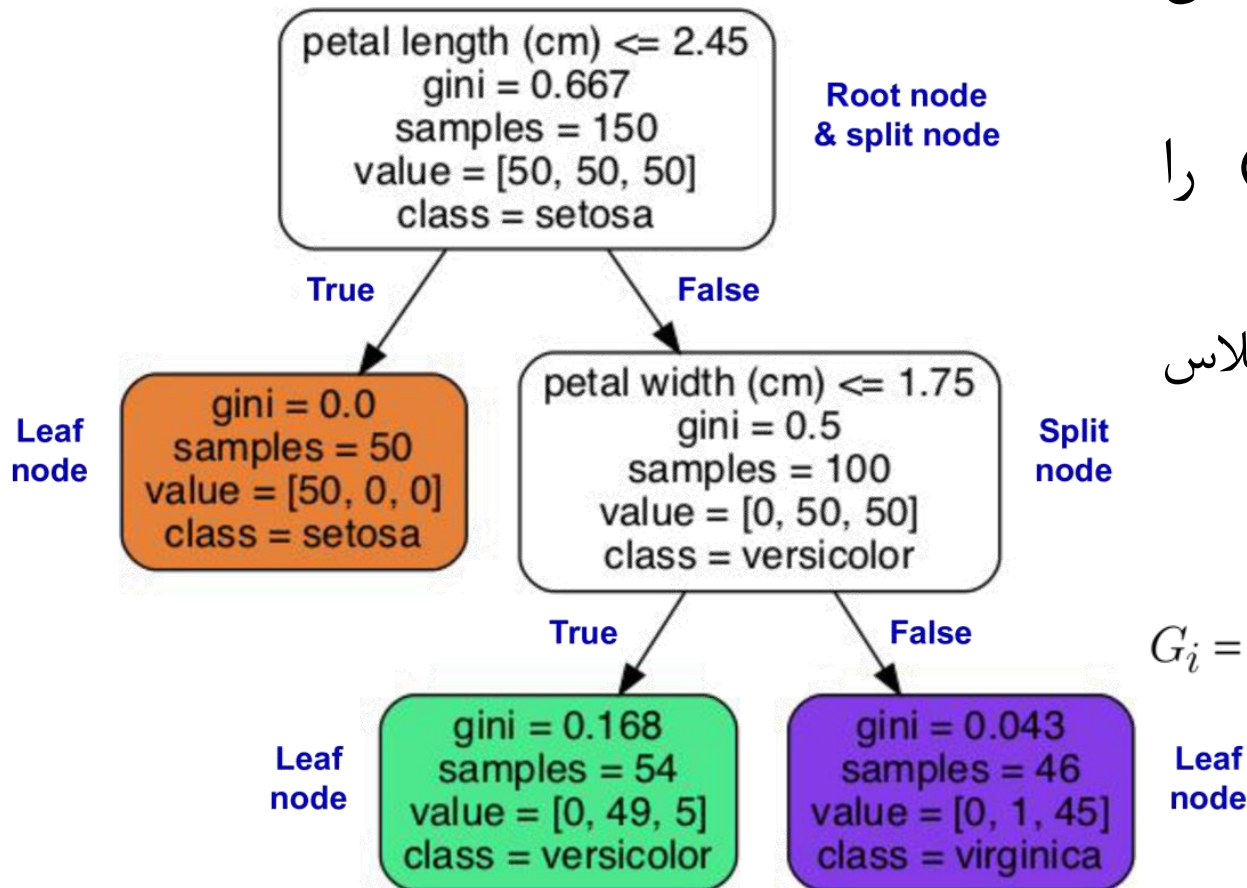
Decision Trees

معیار ناخالصی Gini

- در DT می‌خواهیم به node‌های تا حد امکان خالص دست پیدا کنیم
- نیاز به معیاری داریم که ناخالصی (impurity) را اندازه‌گیری کند
- یک node خالص است اگر تمام نمونه‌های آن از یک کلاس باشند
- معیار Gini به صورت زیر تعریف می‌شود:

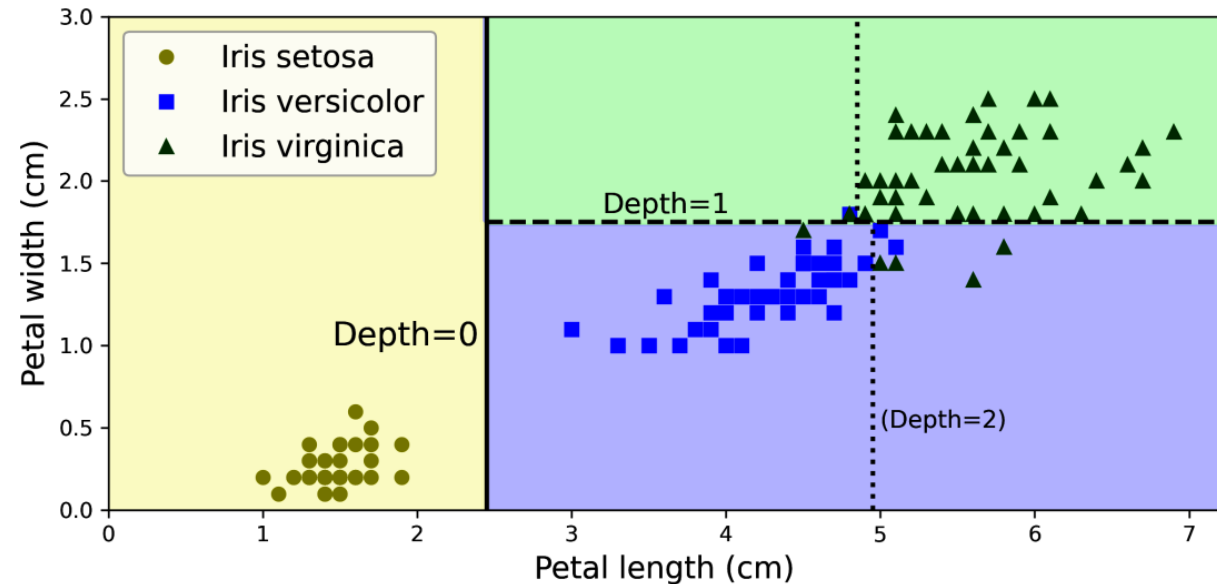
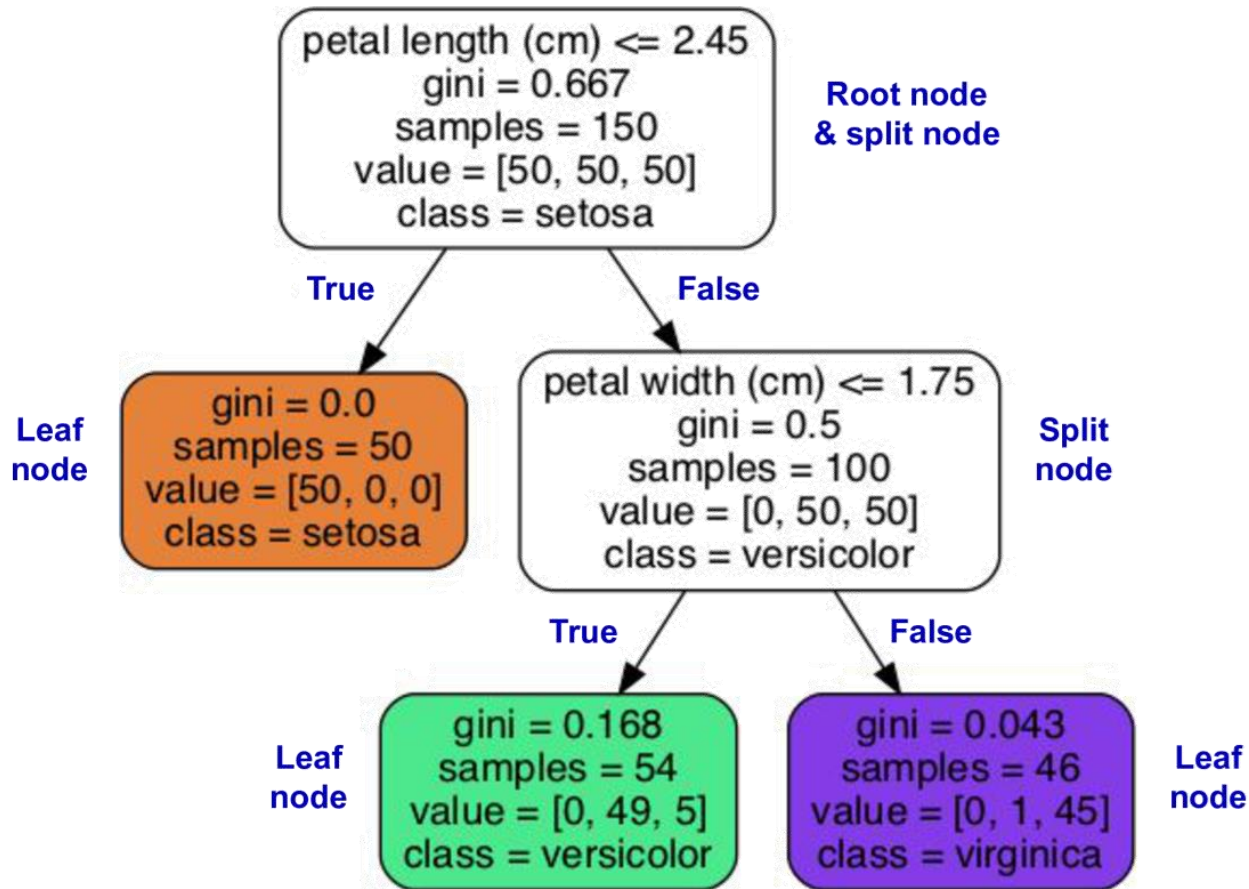
$$G_i = 1 - \sum_{k=1}^n p_{i,k}^2$$

- $p_{i,k}$ درصد تعداد نمونه‌های مربوط به کلاس k در گره i است



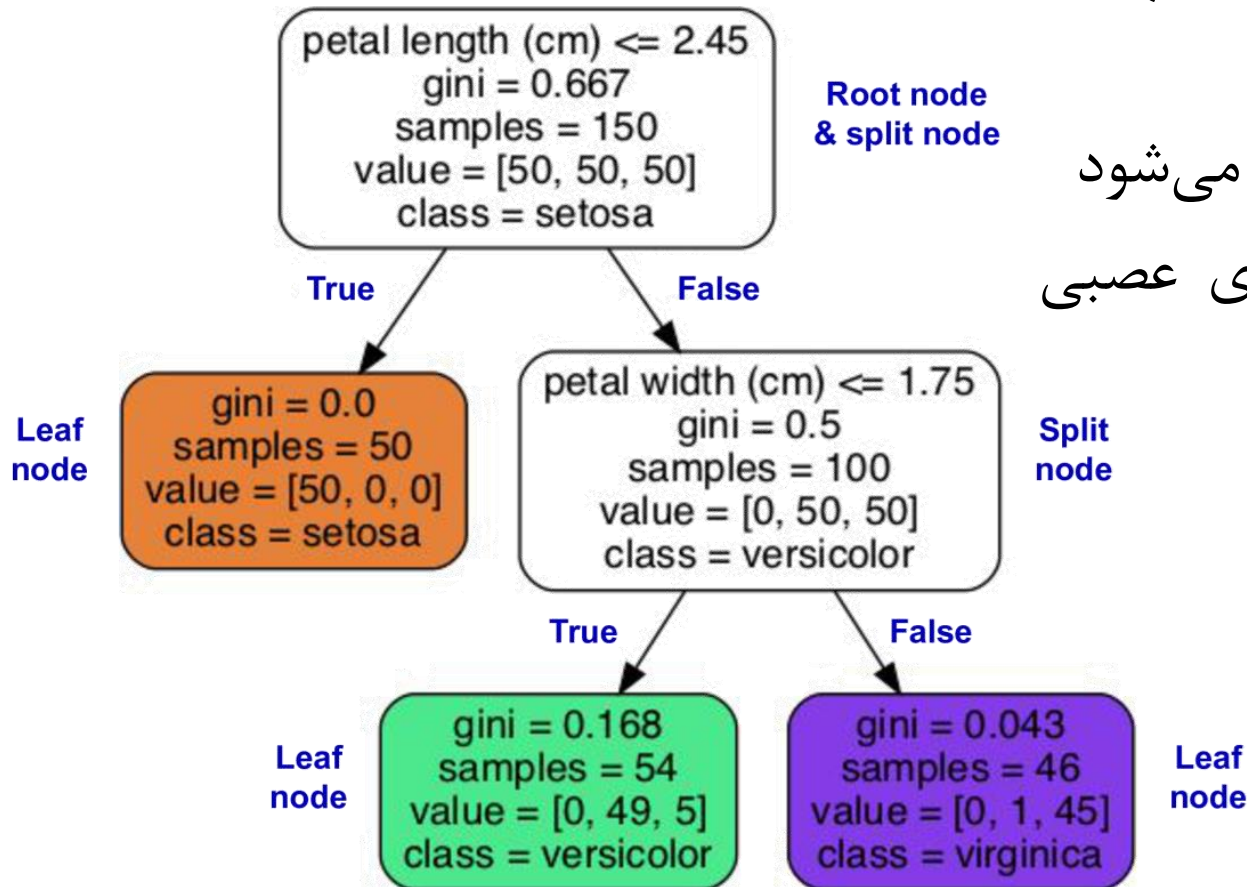
مرز تصمیم

- اگر $\text{max_depth} = 3$ باشد، خط‌چین‌های مربوط به $\text{Depth} = 2$ هم وجود خواهند داشت



تفسیر مدل (Model Interpretation)

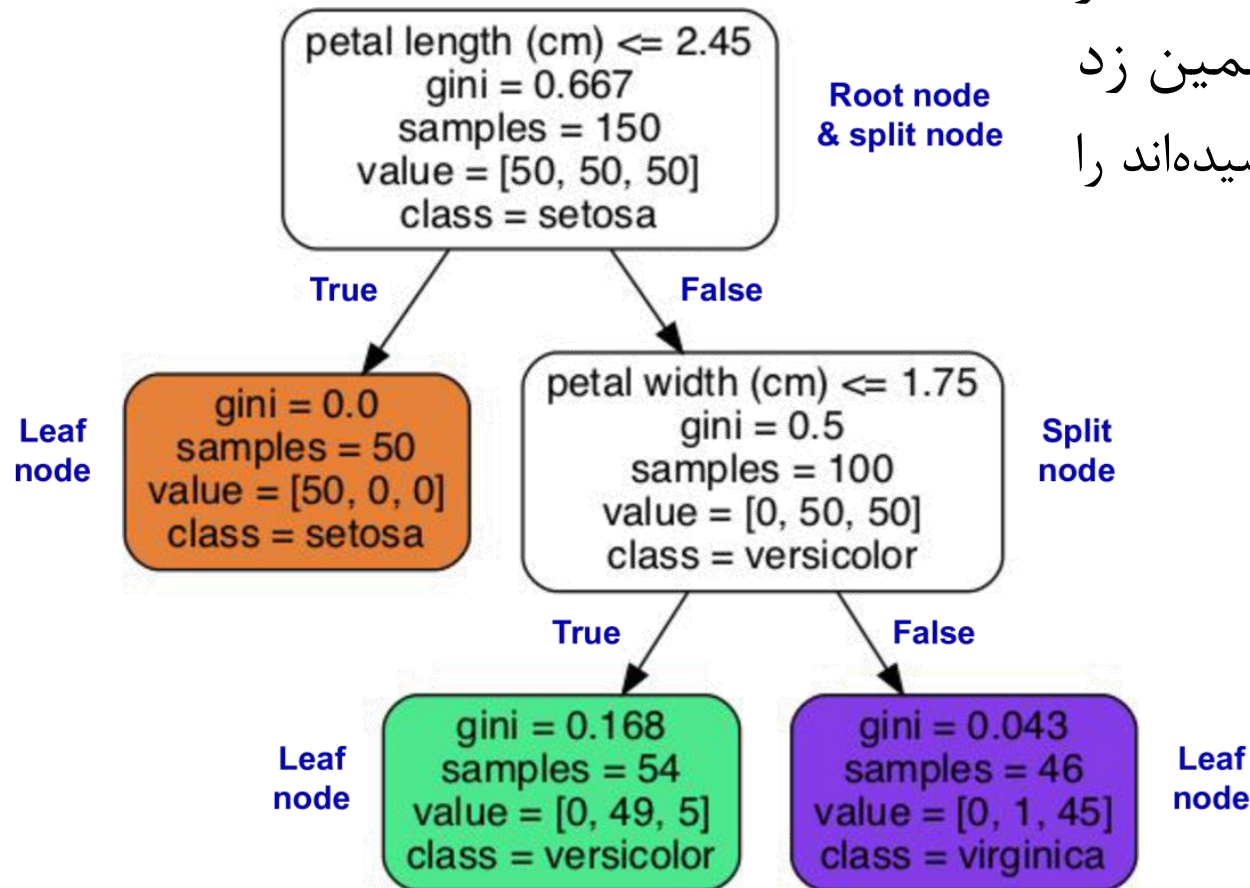
- درختان تصمیم بسیار واضح هستند و تصمیمات آنها به سادگی قابل تفسیر است
- به چنین مدل‌هایی جعبه سفید (white box) گفته می‌شود
- در مقابل، به مدل‌های پیچیده‌تری مانند شبکه‌های عصبی جعبه سیاه (black box) گفته می‌شود
- یادگیری ماشین تفسیرپذیر (Interpretable ML) یک زمینه تحقیقاتی بسیار فعال است که تلاش می‌کند تصمیم گرفته شده توسط مدل را به نحوی که برای انسان قابل فهم باشد بیان کند



تخمین احتمال کلاس‌ها

- در DT به سادگی می‌توان احتمال تعلق یک نمونه به هر کلاس را بر اساس گره‌ای که در آن قرار می‌گیرد تخمین زد - می‌توان درصد نمونه‌هایی از هر کلاس که به این گره رسیده‌اند را به عنوان احتمال آن کلاس استفاده کرد

```
>>> tree_clf.predict_proba([[5, 1.5]]).round(3)
array([[0.    , 0.907, 0.093]])
>>> tree_clf.predict([[5, 1.5]])
array([1])
```



الگوریتم یادگیری CART

- الگوریتم CART (Classification and Regression Tree) برای آموزش درخت‌های باینری استفاده می‌شود
- در گام اول با استفاده از یک ویژگی و یک حد آستانه t_k ، مجموعه آموزشی به دو بخش تقسیم می‌شود
 - مانند $\text{petal length} \leq 2.45 \text{ cm}$
 - چطور k و t_k را انتخاب کنیم؟
 - می‌توان زوج (k, t_k) را جستجو کرد که منجر به خالص‌ترین بخش‌ها شود
 - $G_{\text{left/right}}$ مقدار ناخالصی Gini مربوط به زیرمجموعه left/right است
 - $m_{\text{left/right}}$ تعداد نمونه‌های زیرمجموعه left/right است
- در گام‌های بعد، با همین منطق مجموعه نمونه‌های مربوط به هر گره به دو زیرمجموعه تقسیم می‌شود و این کار به صورت بازگشتی تکرار می‌شود

الگوریتم یادگیری CART

$$J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}$$

- زمانی متوقف می شود که:

- نتواند تقسیمی بیابد که منجر به کاهش ناخالصی شود
- به حداکثر عمق برسد: `max_depth`
- و بسیاری شرایط دیگر برای جلوگیری از `overfitting`

```
class sklearn.tree.DecisionTreeClassifier(*, criterion='gini', splitter='best',  
max_depth=None, min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0,  
max_features=None, random_state=None, max_leaf_nodes=None, min_impurity_decrease=0.0,  
class_weight=None, ccp_alpha=0.0)
```

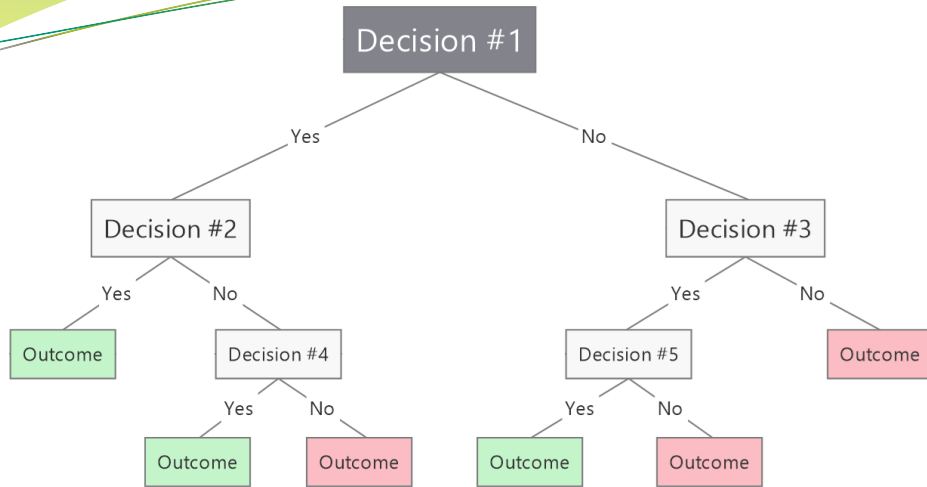
[\[source\]](#)

- CART یک الگوریتم حریصانه (greedy) است که در بسیاری از موارد پاسخ مناسبی را بدست می آورد که لزوماً بهترین پاسخ نیست

ابریارامترهای منظم سازی

- درخت‌های تصمیم فرضیات بسیار محدودی راجع به داده‌های آموزشی دارند و می‌توانند هر تابع پیچیده‌ای را یاد بگیرند و امکان بیش‌برازش آنها بسیار زیاد است
- در یک مدل پارامتری، مانند مدل خطی، فرضیاتی راجع به مدل وجود دارد و تعداد محدودی پارامتر دارند که درجه آزادی آن را محدود و خطر بیش‌برازش را کاهش می‌دهد
- درخت تصمیم هم ابریارامترهای زیادی برای جلوگیری از بیش‌برازش دارد

ابریارامترهای منظم‌سازی



- `max_depth`: حداکثر عمق درخت
- `max_leaf_nodes`: حداکثر تعداد برگ‌ها
- `min_samples_leaf`: حداقل تعداد نمونه‌های یک برگ
- `min_weight_fraction_leaf`: مشابه با `min_samples_leaf` اما به صورت درصد از کل نمونه‌ها
- `min_samples_split`: حداقل تعداد نمونه‌های یک گره برای آنکه تقسیم شود
- `max_features`: حداکثر تعداد ویژگی‌هایی که در هر گره بررسی می‌شود
- افزایش ابریارامترهای `min_*` و کاهش ابریارامترهای `max_*` مدل را منظم خواهد کرد
- برخی الگوریتم‌های دیگر از هرس کردن (pruning) درخت پس از آموزش استفاده می‌کنند

```
from sklearn.datasets import make_moons
```

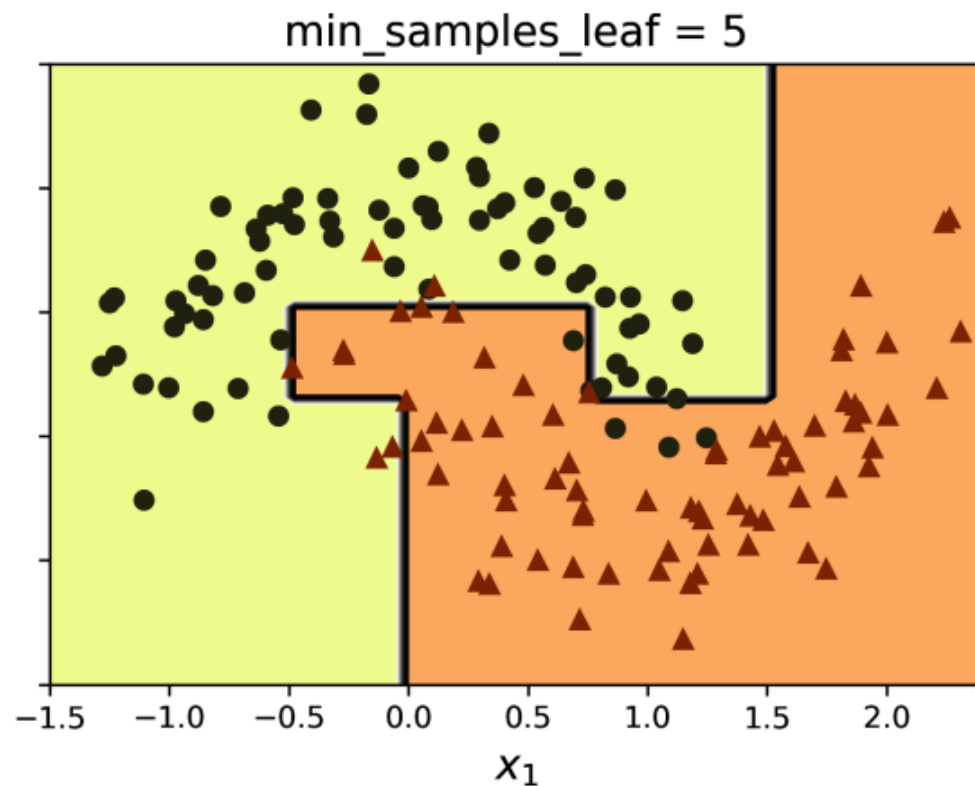
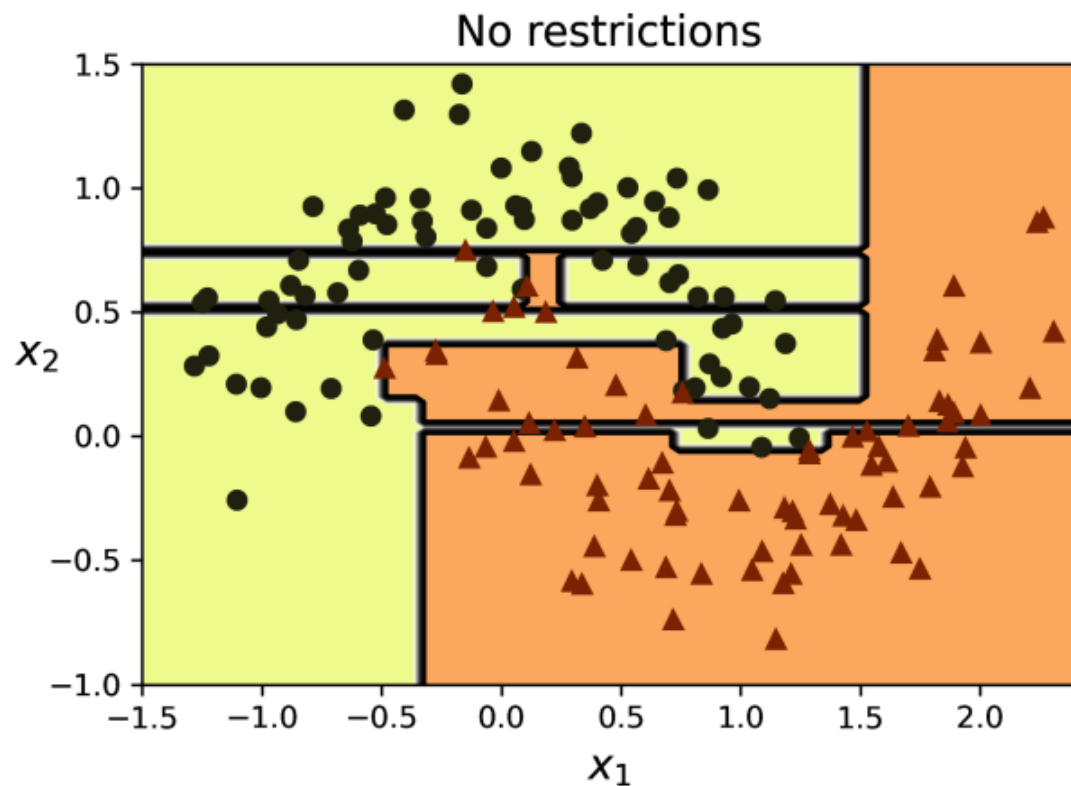
```
X_moons, y_moons = make_moons(n_samples=150, noise=0.2, random_state=42)
```

```
tree_clf1 = DecisionTreeClassifier(random_state=42)
```

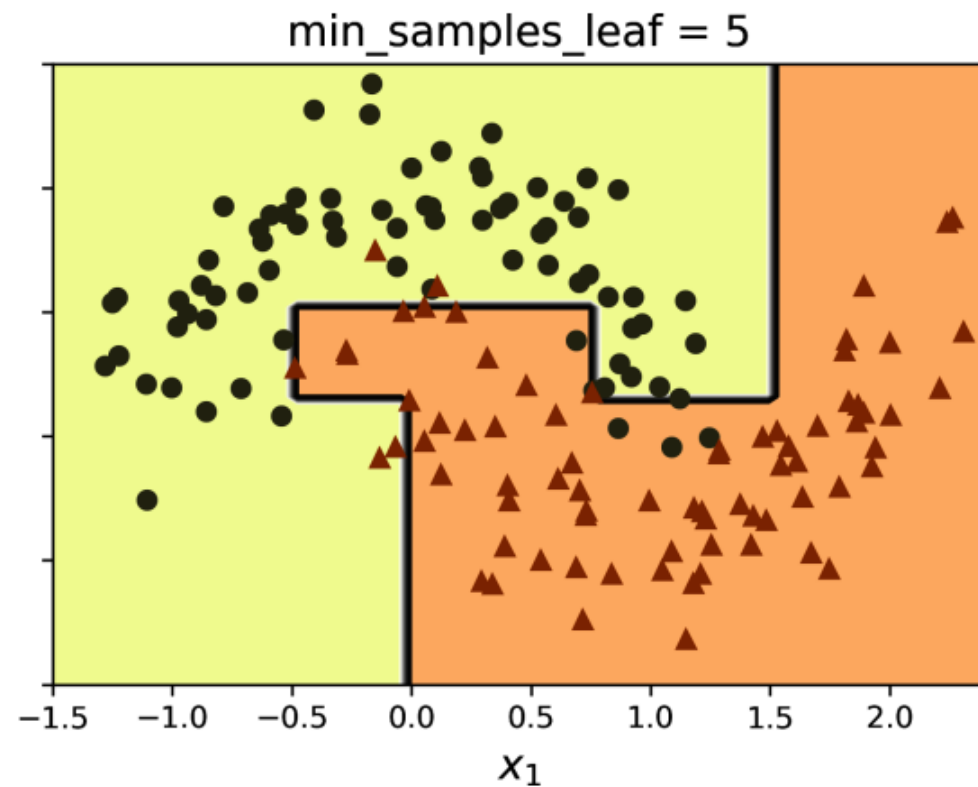
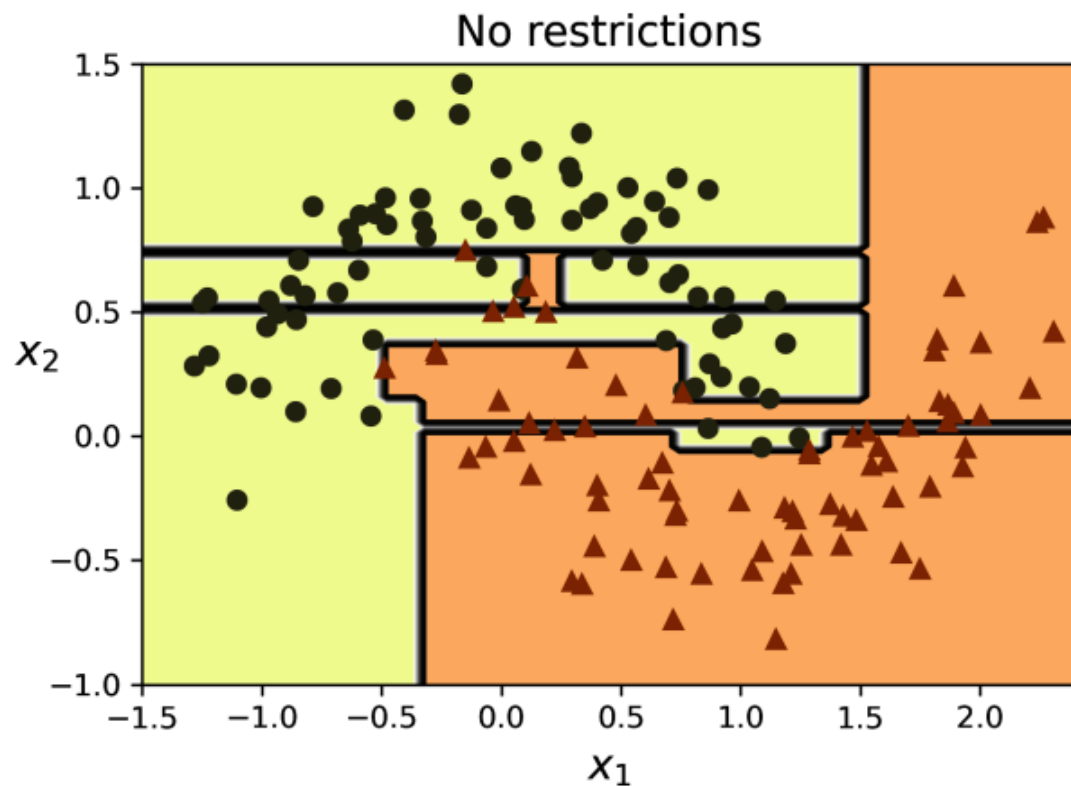
```
tree_clf2 = DecisionTreeClassifier(min_samples_leaf=5, random_state=42)
```

```
tree_clf1.fit(X_moons, y_moons)
```

```
tree_clf2.fit(X_moons, y_moons)
```



```
>>> X_moons_test, y_moons_test = make_moons(n_samples=1000, noise=0.2,  
...                                          random_state=43)  
...  
...  
>>> tree_clf1.score(X_moons_test, y_moons_test)  
0.898  
>>> tree_clf2.score(X_moons_test, y_moons_test)  
0.92
```



رگرسیون

- درخت تصمیم می‌تواند به سادگی برای رگرسیون هم استفاده شود

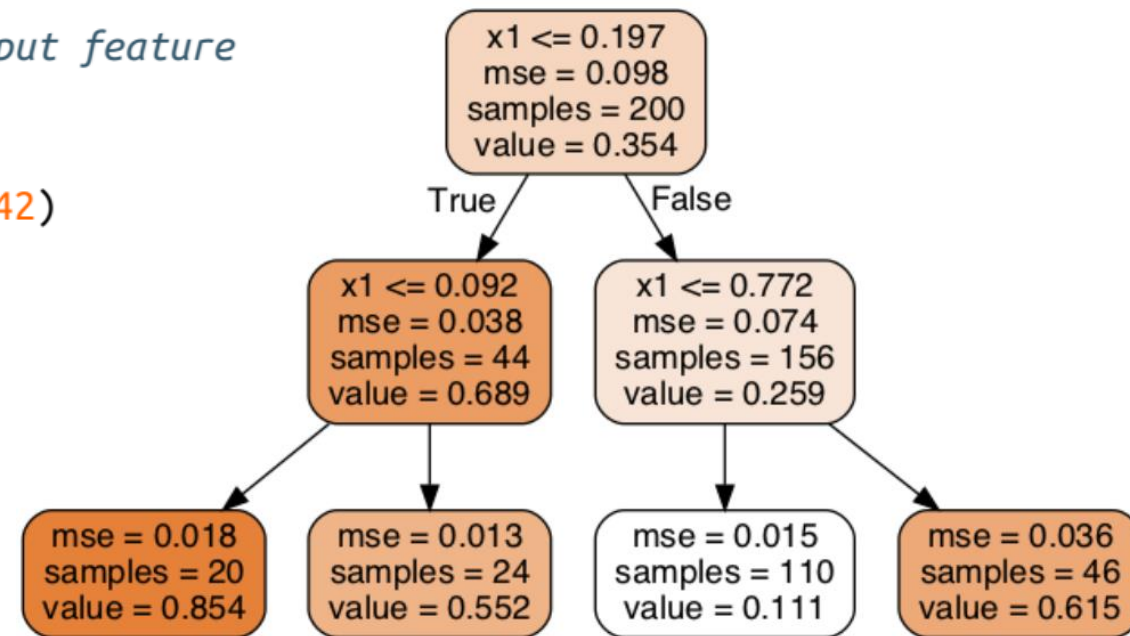
- به عنوان مثال، در هر گره یک عدد ثابت را پیش‌بینی کند

- مثال: $\hat{y} = 0.111 \Leftarrow x_1 = 0.2$

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor

np.random.seed(42)
X_quad = np.random.rand(200, 1) - 0.5 # a single random input feature
y_quad = X_quad ** 2 + 0.025 * np.random.rand(200, 1)

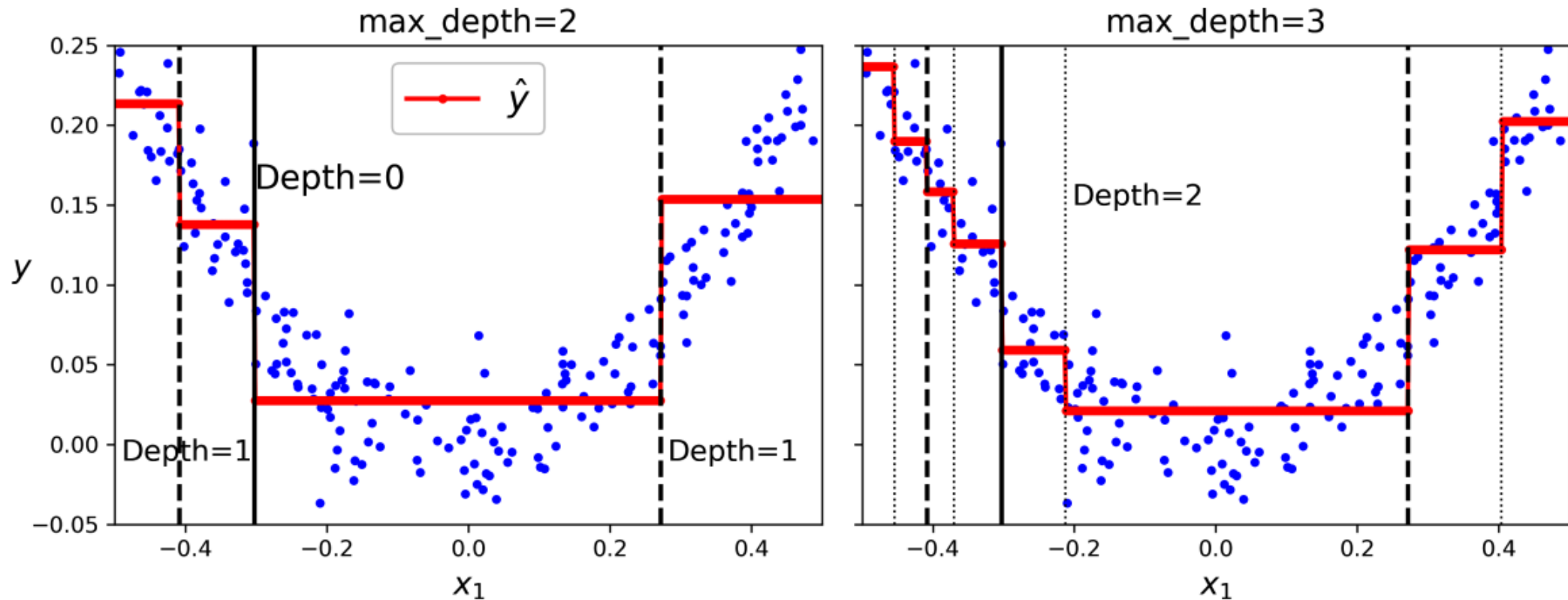
tree_reg = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg.fit(X_quad, y_quad)
```



```
import numpy as np
from sklearn.tree import DecisionTreeRegressor
```

```
np.random.seed(42)
X_quad = np.random.rand(200, 1) - 0.5 # a single random input feature
y_quad = X_quad ** 2 + 0.025 * np.random.randn(200, 1)
```

```
tree_reg = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg.fit(X_quad, y_quad)
```

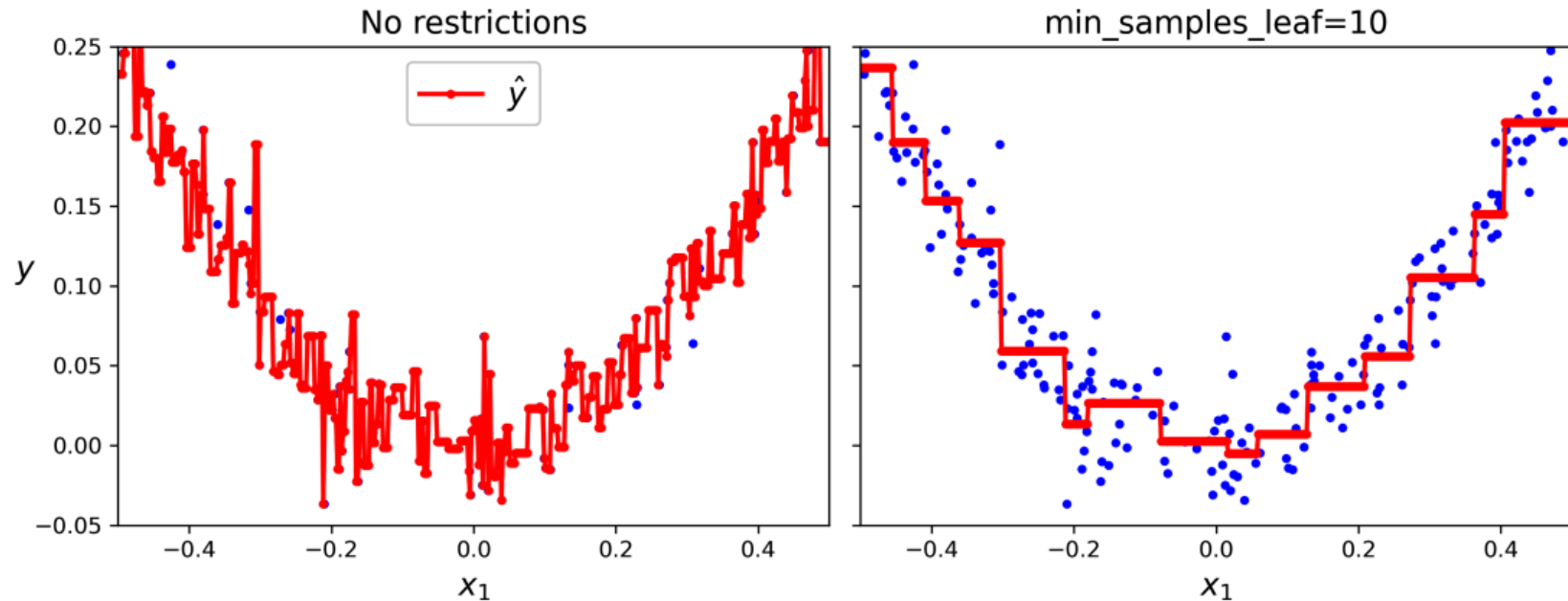


الگوریتم یادگیری CART

- برای رگرسیون، بجای معیار Gini، از معیار MSE استفاده می شود
- برای جلوگیری از بیش برآزش، نیاز به منظم سازی دارد

$$J(k, t_k) = \frac{m_{\text{left}}}{m} \text{MSE}_{\text{left}} + \frac{m_{\text{right}}}{m} \text{MSE}_{\text{right}}$$

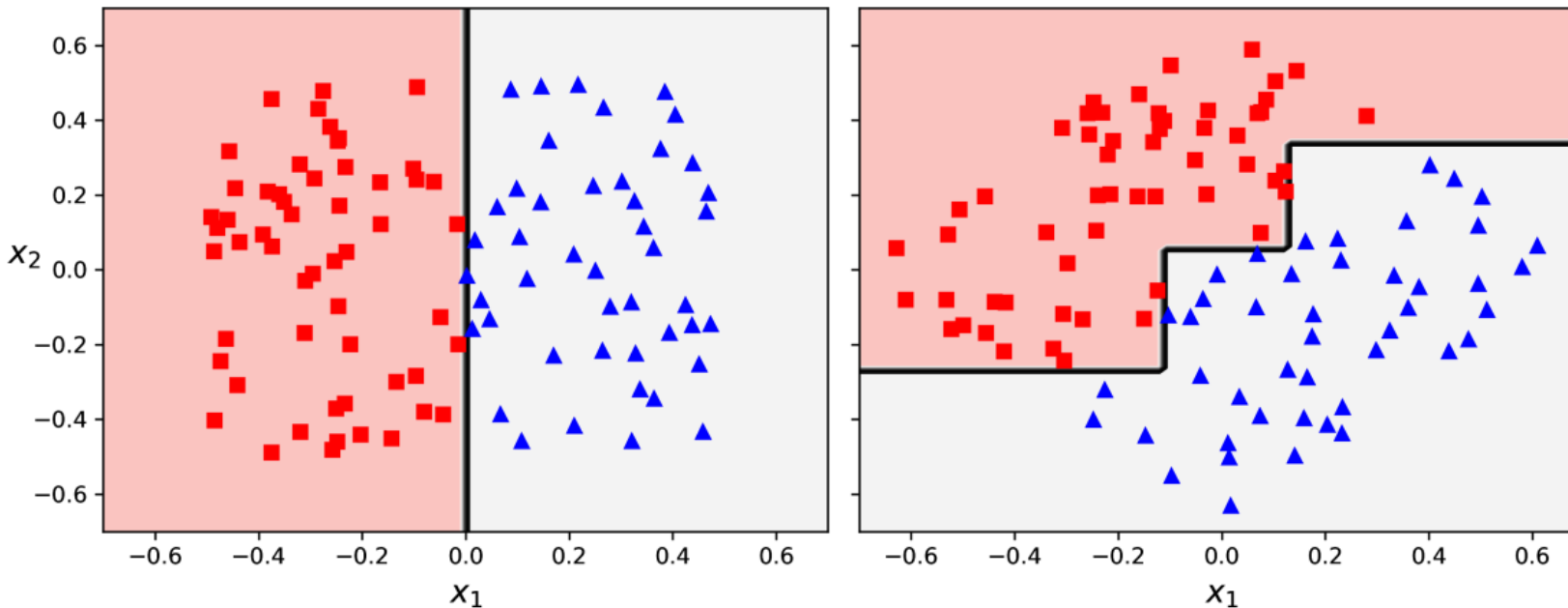
where $\left\{ \begin{array}{l} \text{MSE}_{\text{node}} = \frac{\sum_{i \in \text{node}} (\hat{y}_{\text{node}} - y^{(i)})^2}{m_{\text{node}}} \\ \hat{y}_{\text{node}} = \frac{\sum_{i \in \text{node}} y^{(i)}}{m_{\text{node}}} \end{array} \right.$



حساسیت به جهت محورها

- درخت تصمیم به نسبت ساده و قابل تفسیر، همه کاره و قدرتمند است
- با این حال، محدودیتهایی هم دارد
- در درخت تصمیم مطلوب این است که مرزهای تصمیم عمود بر محورهای مختصات باشند

- داده ساده زیر را در نظر بگیرید
- اگر داده‌ها ۴۵ درجه بچرخند؟
- تعمیم‌دهی ضعیف‌تری دارد

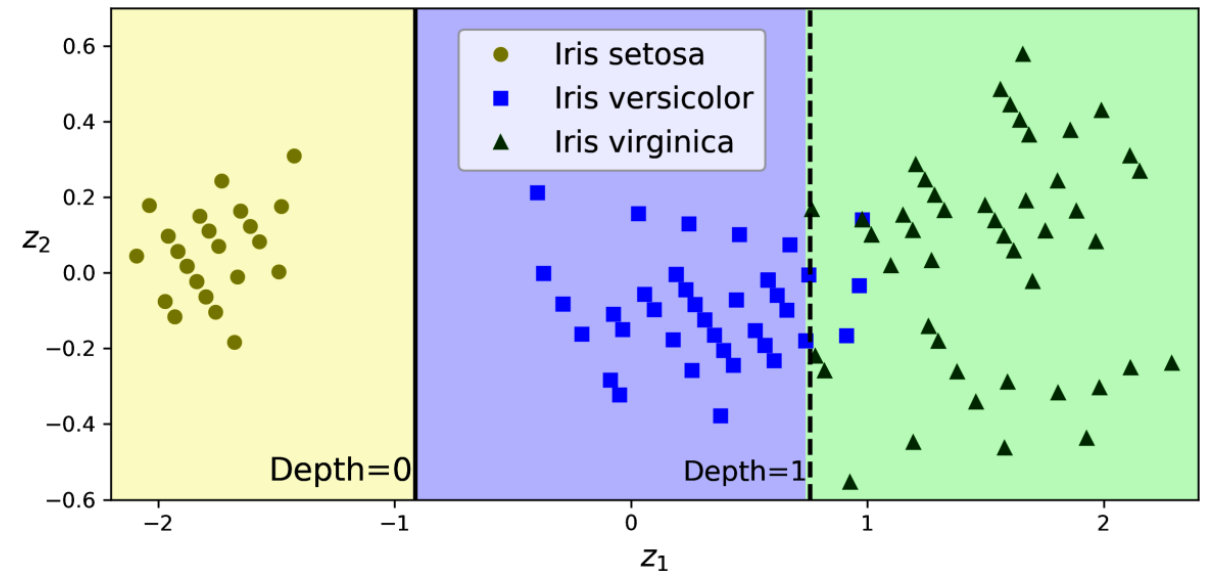
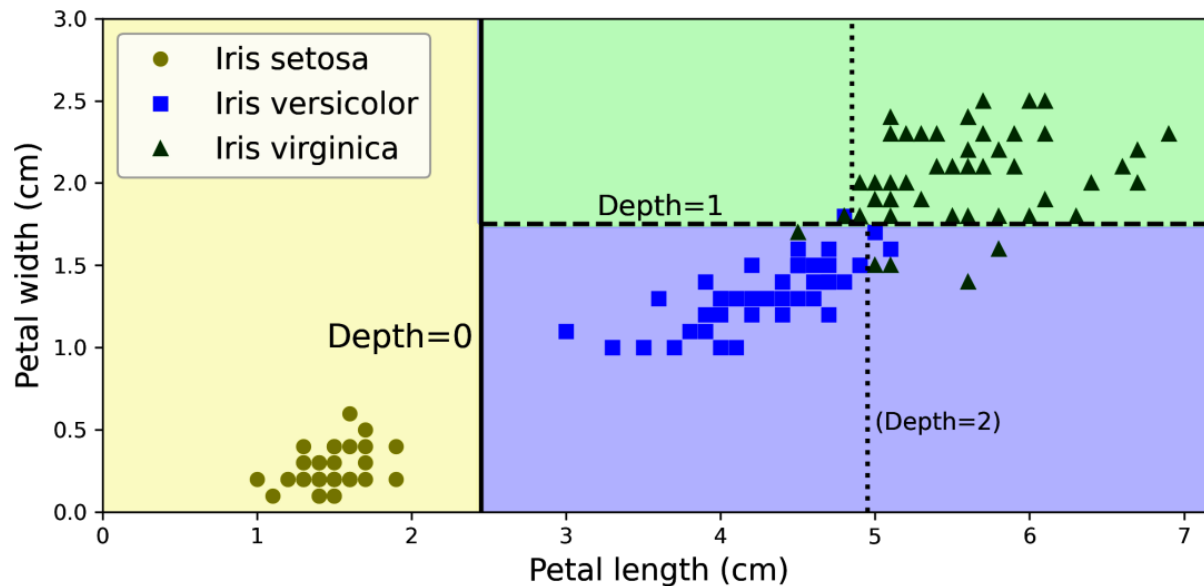


حساسیت به جهت محورها

```
from sklearn.decomposition import PCA
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
```

```
pca_pipeline = make_pipeline(StandardScaler(), PCA())
X_iris_rotated = pca_pipeline.fit_transform(X_iris)
tree_clf_pca = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf_pca.fit(X_iris_rotated, y_iris)
```

- می‌توان با کمک برخی تبدیل‌های هندسی، جهت ویژگی‌ها را اصلاح کرد



یادگیری گروهی و جنگل تصادفی

Ensemble Learning and Random Forests

یادگیری گروهی

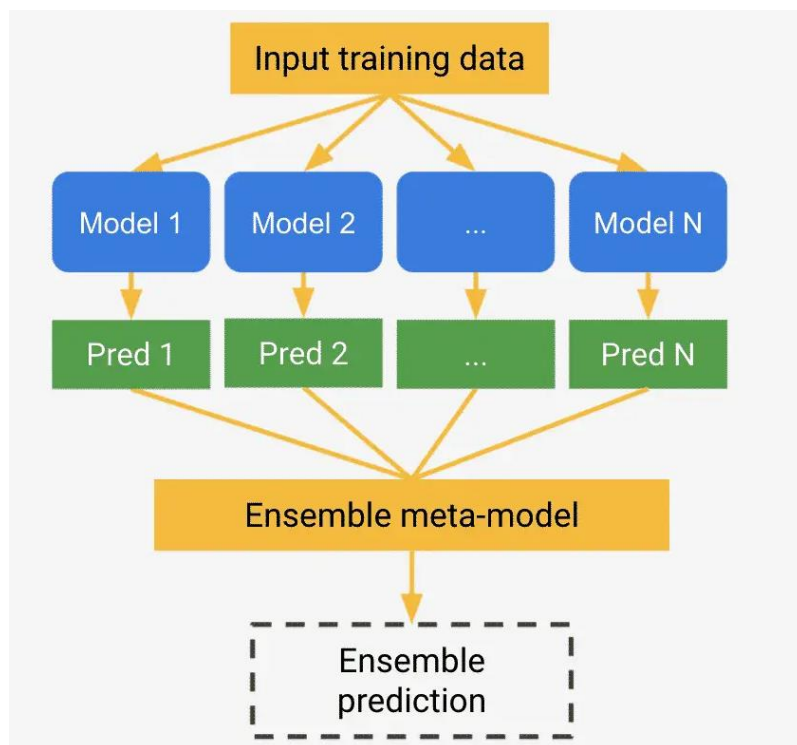
- فرض کنید یک سوال پیچیده از هزاران انسان به صورت تصادفی پرسیده شود و پاسخ آنها تجميع شود
- در بسیاری از مواقع، این پاسخ از پاسخ یک انسان خبره هم بهتر است

- حکمت جمعیت (wisdom of the crowd)

- در یادگیری ماشین هم می توان پیش بینی های گروه هایی از پیش بینی کننده ها را تجميع کرد

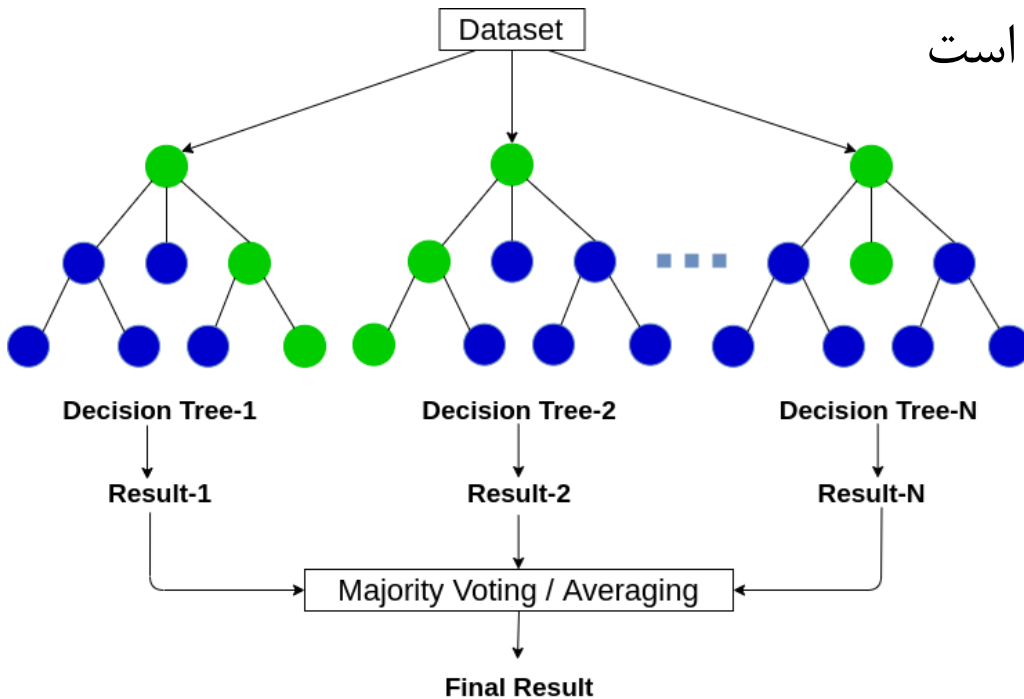
- اغلب پیش بینی های بهتری نسبت به بهترین پیش بینی کننده فردی خواهد داشت

- گروهی از پیش بینی کننده ها یک ensemble نامیده می شوند



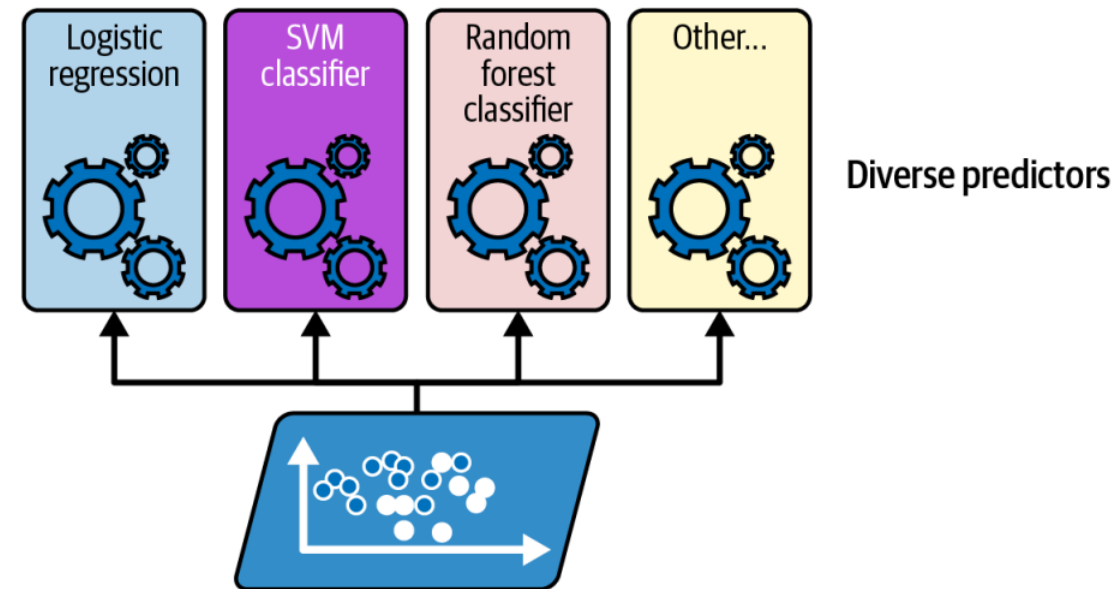
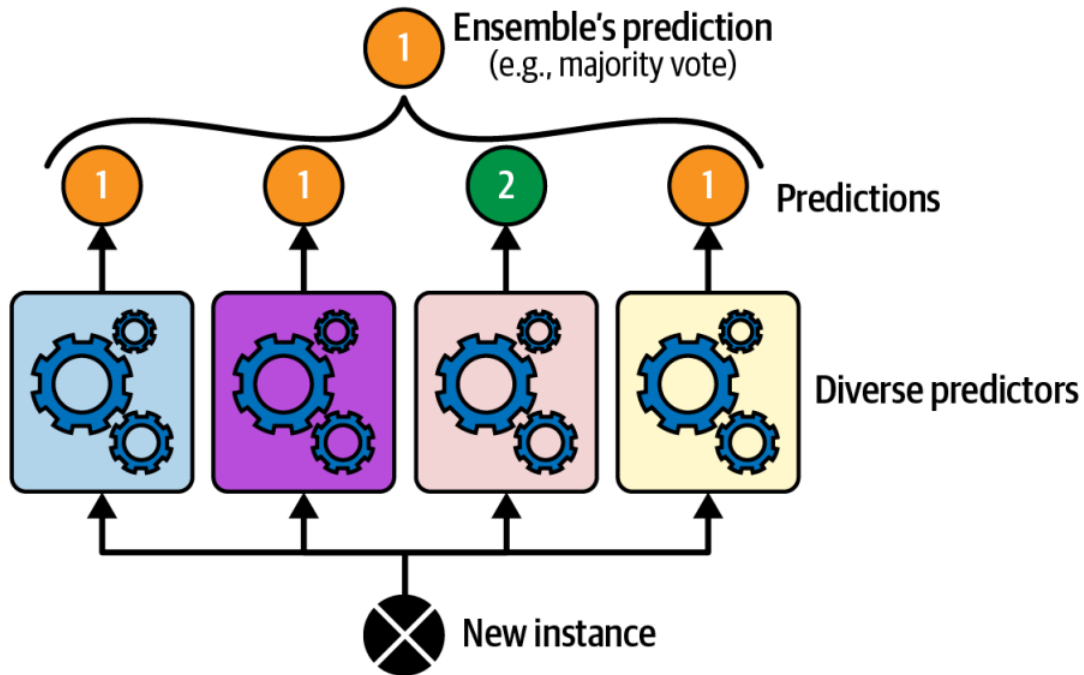
جنگل تصادفی

- یک مثال معروف از یادگیری گروهی، جنگل تصادفی (random forest) است که تعدادی درخت تصمیم با شرایط مختلف آموزش می‌دهد و پیش‌بینی‌های آنها را جمع می‌کند
 - مثلاً برای آموزش هر درخت از یک زیرمجموعه تصادفی از داده‌های آموزشی استفاده شود
 - با وجود سادگی، یکی از قدرتمندترین الگوریتم‌های یادگیری ماشین است



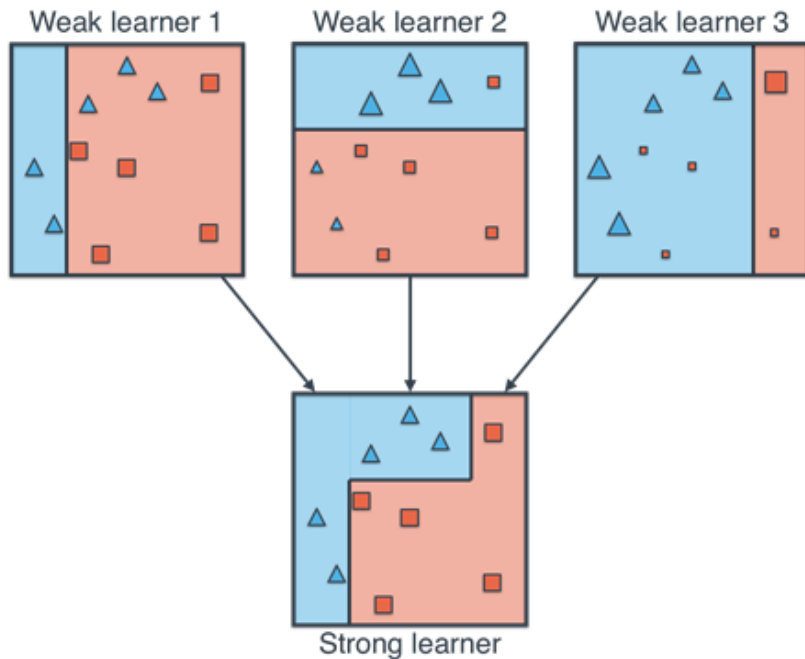
دسته‌بندی مبتنی بر رأی‌گیری

- پس از دستیابی به دسته‌بندی‌هایی که عملکرد نسبتاً خوبی برای مسئله مورد نظر دارند، یک راه بسیار ساده برای تجمیع پیش‌بینی‌های آنها، انتخاب کلاسی است که بیشترین رأی (vote) را دارد
- دسته‌بندی اکثریت آراء (majority-vote) یا دسته‌بندی رأی سخت (hard voting) نامیده می‌شود



دسته‌بندی مبتنی بر رای‌گیری

- دسته‌بندی با رای‌گیری اغلب دقت بهتری نسبت به بهترین دسته‌بند موجود در گروه بدست می‌آورد
- حتی اگر هر کدام از دسته‌بندها یک یادگیرنده ضعیف (weak learner) باشند (کمی بهتر از حدس تصادفی)، نتیجه رای‌گیری می‌تواند یک یادگیرنده قوی (strong learner) باشد اگر یادگیرنده‌های ضعیف تنوع کافی داشته باشند



- قابل اثبات است که اگر از ۱۰۰۰ دسته‌بند ضعیف مستقل با دقت ۵۱٪ رای‌گیری انجام شود، دقت ۷۵٪ بدست می‌آید
- با ۱۰۰۰۰ می‌توان با بالای ۹۷٪ رسید
- معمولاً دسته‌بندها بر روی داده‌های مشابهی آموزش می‌بینند و مستقل نیستند!

```

from sklearn.datasets import make_moons
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

X, y = make_moons(n_samples=500, noise=0.30, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

```

```

voting_clf = VotingClassifier(
    estimators=[
        ('lr', LogisticRegression(random_state=42)),
        ('rf', RandomForestClassifier(random_state=42)),
        ('svc', SVC(random_state=42))
    ]
)
voting_clf.fit(X_train, y_train)

```

```

>>> for name, clf in voting_clf.named_estimators_.items():
...     print(name, "=", clf.score(X_test, y_test))
...

```

```

lr = 0.864
rf = 0.896
svc = 0.896

```

```

>>> voting_clf.score(X_test, y_test)
0.912

```

```

>>> voting_clf.predict(X_test[:1])
array([1])
>>> [clf.predict(X_test[:1]) for clf in voting_clf.estimators_]
[array([1]), array([1]), array([0])]

```

رای گیری نرم

- می توان در رای گیری از احتمال هر کلاس هم استفاده کرد (soft voting)
 - رای هر مدلی که از پیش بینی خود مطمئن تر است، وزن بیشتری داشته باشد
 - مدل ها باید بتوانند احتمال هم پیش بینی کنند

```
>>> voting_clf.score(X_test, y_test)
0.912
```

```
>>> voting_clf.voting = "soft"
>>> voting_clf.named_estimators["svc"].probability = True
>>> voting_clf.fit(X_train, y_train)
>>> voting_clf.score(X_test, y_test)
0.92
```